

Solution Requirements

Date	04 July 2026
Team ID	ramanharshitha2005@gmail.com
Project Name	OptiCrop - Smart Agricultural Production Optimization Engine
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

- Functional Requirements (FR) define specific behaviors, functions, or features of the system (What the system should do).
- Non-Functional Requirements (NFR) specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should be).

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	No login required. OptiCrop is an open, local web app accessible to any user who runs server.py on their machine (http://127.0.0.1:8000). No authentication layer is implemented by design to maximise accessibility for farmers.	Low
2	Authorization Levels	Single-user access — all features are available to everyone who runs the application. No role-based access control. Future versions may add researcher vs. farmer roles.	Low
3	External Interfaces	Open-Meteo Geocoding API (https://geocoding-api.open-meteo.com/v1/search) and Open-Meteo Forecast API (https://api.open-meteo.com/v1/forecast). No API key required. Used for real-time temperature, humidity, and 30-day rainfall auto-fill.	High
4	Transactions / Core Processing	POST /api/recommend — accepts 9 features (N, P, K, temp, humidity, pH, rainfall, location, season) and returns predicted crop + confidence using a Random Forest + Decision Tree + SVC ensemble (crop_model.joblib). POST /api/suitability — evaluates 7 environmental metrics against crop dataset profiles and returns a weighted suitability score. POST /api/disease — classifies disease from crop + symptoms + environment. POST /api/image-disease — classifies disease from uploaded leaf image bytes.	High
5	Reporting	GET /api/analytics — returns aggregated statistics by crop, location, and season. The insights.html page visualises this using Chart.js bar, radar, and scatter charts. Users can export filtered data as a CSV file via the Export button.	High
6	Business Rules	Crop recommendation uses an ensemble of 3 ML models (RF, DT, SVC) with probability averaging. Suitability scoring uses a weighted formula across 7 metrics (N=20%, P=16%, K=16%, Temp=14%, Humidity=13%, pH=10%, Rainfall=11%). Fertilizer advice adjusts dynamically based on actual NPK values vs. crop-specific targets.	High
7	Compliance to Laws / Regulations	No personal data is collected or stored. All processing is local. Open-Meteo API is used under its free, open licence. The app does not collect, transmit, or store any farmer identity data.	Medium

8	Other	Disease identification supports two modes: (a) symptom + environment input via /api/disease and (b) leaf image upload via /api/image-disease (supports JPEG, PNG, BMP, GIF, TIFF). The server uses Python's built-in ThreadingHTTPServer — no external web framework (Flask/Django) is required.	High
---	-------	--	------

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	All API endpoints must respond within acceptable time. Crop recommendation and suitability endpoints process a single request in under 1 second. Weather API response is network-dependent but typically under 2 seconds.	Crop rec < 1s; Weather < 3s; Image disease < 5s
2	Scalability	Python ThreadingHTTPServer handles concurrent requests via threads. Suitable for single-machine local use with up to ~20 simultaneous users. For larger scale, a WSGI server (gunicorn) and load balancer would be added.	Handles 10–20 concurrent local users
3	Security & Data Privacy	Directory traversal is prevented in safe_resolve(). Only allowed file extensions are served (.html, .css, .js, .json, .png, .jpg, .ico, .svg, .webp). No user data is logged or persisted. API key for weather: none required (Open-Meteo is open).	No path traversal; no PII stored
4	Reliability & Availability	The app runs locally — availability depends on the host machine. Fallback data (opticrop-data.json) is used if Crop_recommendation.csv is not present. The /api/recommend endpoint returns a graceful fallback response if the model file is missing.	Graceful fallback on missing model/data files
5	Usability & Accessibility	Five clearly labelled HTML pages with semantic structure. Mobile-responsive via CSS (hamburger menu on small screens). GPS-based weather auto-fill reduces manual data entry for farmers. Tooltips and labels on every input field.	Mobile-responsive; works in any modern browser
6	Other (Maintainability)	All ML model training scripts are in ml/ folder. server.py is single-file and self-contained. Models are stored as .joblib files — swappable without changing server logic. requirements.txt pins all dependencies for reproducibility.	Modular ML scripts; single-file server